Anime Recommendation System

Author: Chenfeng Wang

School: CSE, The University of New South Wales (UNSW), Sydney, Australia

Abstract

This project builds a practical anime recommender for experienced viewers. We implement three models: a content-based model using TF-IDF with SVD, an item-item collaborative filter, and a two-stage hybrid. The hybrid first uses collaborative filtering to form a candidate set, then re-ranks with content similarity. We evaluate with standard Top-N metrics and adopt Top-10 as the primary target because it fits the UI and preserves precision. Improvements come from feature engineering and tuning, not from switching datasets. We discuss trade-offs, computational cost, and realistic extensions.

1. Problem

Anime platforms face severe choice overload. The catalogue is large, genres are diverse, and users quickly tire of browsing. Our goal is to present a short, personalized list that users can trust and act on within seconds.

1.1 System deployment and flows

The recommender runs as a service consumed by web and mobile clients behind a feature flag. At login we fetch the user profile, retrieve candidates, re-rank, and render a Top-10 row. We log impressions to enable later analysis. After a user completes a show, we refresh the row and surface Next-Up suggestions. If a profile is missing, we fall back to genre-based onboarding and a light popularity prior.

1.2 Inputs, outputs, triggers, and feedback

Type	Signal	When collected	Used for
Input	Explicit ratings (1–10)	Whenever available	Stable preference signal

Input	Like / Dislike on cards	Inline in UI	Fast profile updates
Input	Onboarding genres	First session	Cold-start handling
Output	Ranked Top-10 with short reasons	Login / refresh / after finish	Decision support
Trigger	Push reminders	Daily or weekly	Re-engagement
Feedback	Clicks, saves, dwell time	Continuous	Monitoring and future learning

1.3 Problem framing

We frame the task as ranking. Collaborative filtering predicts scores internally, content models compute similarities, and the system outputs a ranked list. For evaluation, we treat ratings ≥ 8 in the hold-out set as relevant items.

1.4 Competitor analysis and gaps

Crunchyroll/VRV relies on ALS over watch data with frequent retraining and hybrid blocks (Erdogan & Narayanan, 2020). It scales and surfaces serendipity, but it leans toward popularity and struggles with true cold-start. MyAnimeList provides simple co-audience suggestions on item pages; this is fast but not personalized. Netflix uses an ensemble ranker (Gomez-Uribe & Hunt, 2015), emphasizing hybrid designs and diversity. Our gap: strong content understanding for new items plus a simple, explainable hybrid that remains fast.

Platform	Recommendation Approach	Limitations
MyAnimeList	Popularity ranking + basic tag/genre filtering	– Cold-start for new users/items – No deep content semantics
Crunchyroll	Editorial picks + trending & "most watched" lists	– Limited personalization – Favors only latest simulcasts
AniList	User-curated seasonal lists + collaborative lists	– Emphasizes popularity over individual taste
Netflix (anime)	Proprietary hybrid system (CF + content + metadata)	– Closed-source – Not optimized for niche anime sub-genres

Figure 1. Competitor Analysis.

1.5 UI details and micro-interactions

Each row displays 10 concise cards: title, type, episodes, a few genre tags, and a one-line synopsis. Two actions—Like and Dislike—enable quick feedback. A small explanation chip (e.g., “Because you liked Violet Evergarden”) increases trust. We de-duplicate by franchise (max two per list) and keep one novelty slot when scores are very close.

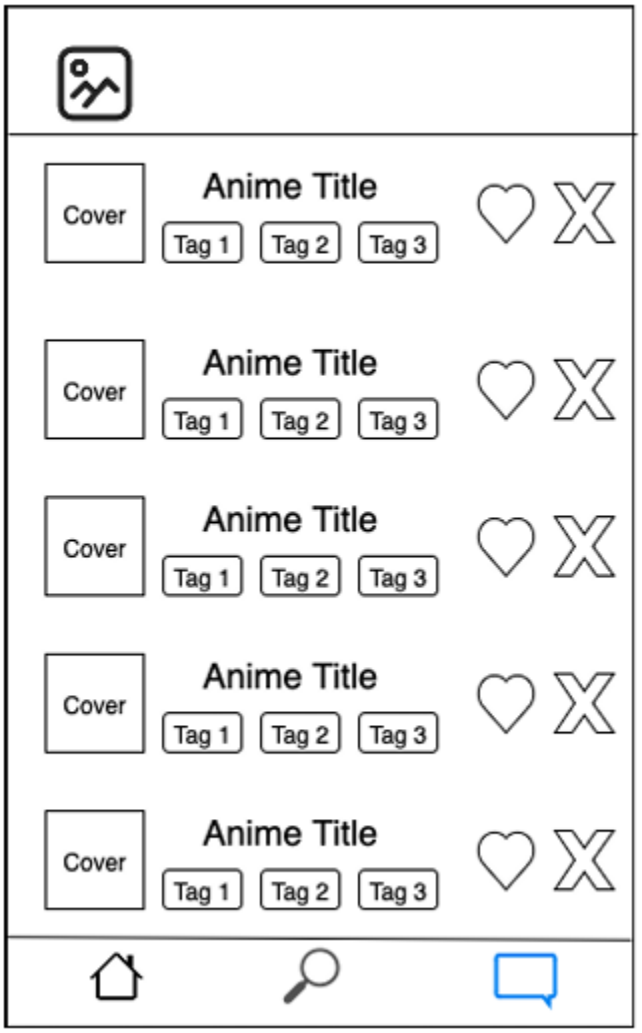

Figure 2. UI.

1.6 Explainability

We generate short, privacy-safe reasons derived from item metadata and aggregate signals, never revealing individual user information. Explanations focus on genre/tone similarity or widely-shared co-audience patterns.

2. Dataset(s)

2.1 Files, keys, joins

File	Key fields	Purpose
anime.csv	MAL_ID, Name, Genres, Type, Episodes, Studios, Source, Rating, Members	Item metadata
anime_with_synopsis.csv	MAL_ID, Synopsis, Genres	Text features
rating_complete.csv	user_id, anime_id (MAL_ID), rating 1–10	Explicit feedback

2.2 Cleaning and normalization

We lowercase text, remove boilerplate like "Source: Manga," strip URLs, and normalize whitespace. We unify genre tags (e.g., merging "Sci-Fi" and "Science Fiction"). Numeric outliers are clamped to plausible ranges. Items lacking synopsis fall back to metadata-only features.

2.3 Exploratory analysis (narrative)

Ratings skew high (mean ≈ 7), which is typical for fan communities. User activity is long-tailed—many rate a handful of titles while a minority rate hundreds. Item popularity is also long-tailed: a few blockbusters dominate attention, while most titles remain niche. Synopses are short and uneven, so raw TF-IDF is sparse and noisy; SVD helps recover stable themes.

2.4 Strengths and weaknesses for the task

Aspect	Strength	Weakness	Mitigation
Scale	Large rating graph	High sparsity	Cosine similarity; SVD

Content	Genres + synopsis	Short / noisy text	Aggressive cleaning; LSA
Labels	Explicit ratings	High-score bias	Use relevance threshold
Freshness	Rich snapshot	Limited temporal data	Warm-start design; sequential next

2.5 Subsets and justification

We focus on active users (≥ 20 ratings; often ≥ 50 for stability). For heavy tests, we sample up to 100,000 users using a fixed seed (42) and keep identical splits across methods to ensure fairness. A top-500 popularity pool is used only for latency stress tests. Crucially, performance gains come from feature engineering and parameter tuning—not from changing the dataset.

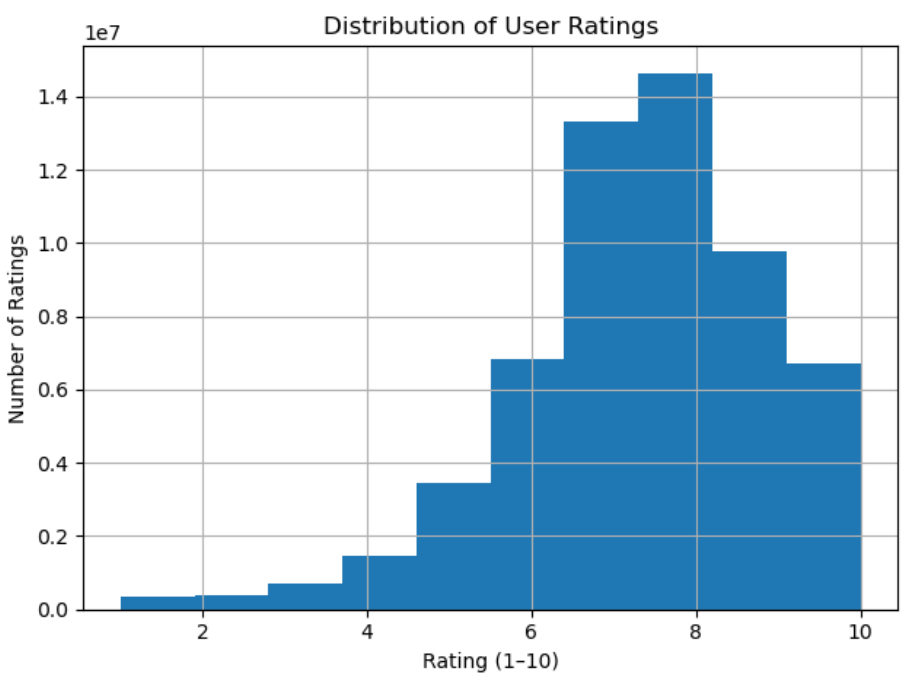

Figure 3. Distribution of User Ratings

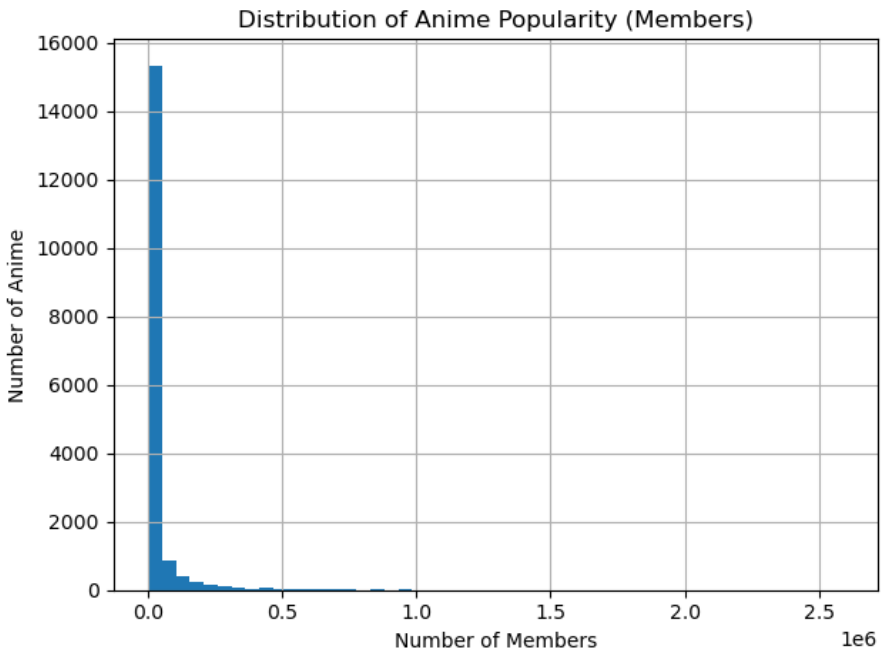

Figure 4. Distribution of Anime Popularity (Members)

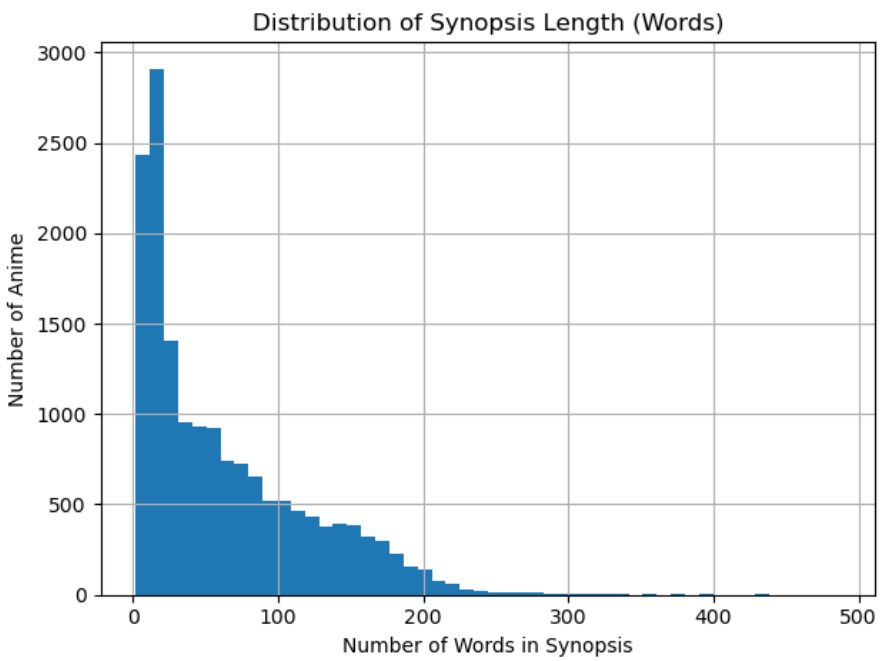

Figure 5. Distribution of Synopsis Length (Words)

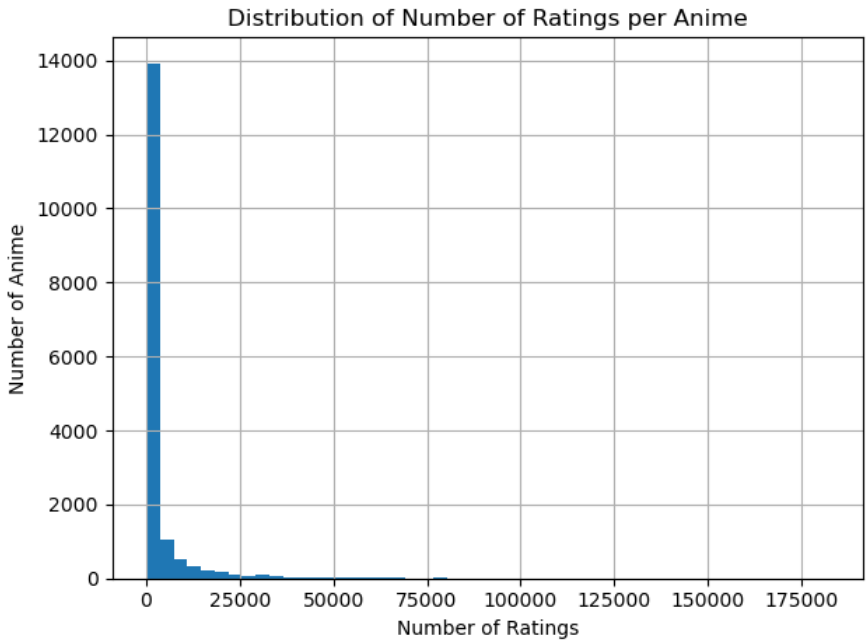

Figure 6. Distribution of Number of Ratings per Anime

3. Method(s)

3.1 Content-based (CB)

We make one text field for each item. Just put all info together: genres, type, studios, producers, source, season, episodes, rating band, and synopsis. Before that, we tag each part, like genres=xxx. This way we know which is which. All letters go to lowercase. We delete useless stuff like URLs. Similar genre names are merged.

Then we use TF-IDF. Only 1-gram and 2-gram. Min_df=5, max_df=0.6. After that, the matrix is big and sparse, so we cut it down with TruncatedSVD (200 comps). Random_state=42. This makes dense vectors. They are smaller, cleaner, and work better for prediction. P@10 is higher than just raw TF-IDF.

We also add three number features: Score, Members, Favorites. All scaled to 0-1. So one item vector has around 203 dims.

User vector is just the average of items they like (rating ≥ 7), weighted by rating. At prediction time, we check cosine similarity between user vector and item vectors. Seen items are removed. If tie, we choose the one with more Members, then higher Score.

Then small diversity check: no too many items from the same franchise. After that, we keep Top-10.

To make it fast, all item vectors are precomputed and cached. User vectors also cached. When new feedback comes, we update user cache right away.

3.1.1 Design choices

Bigrams capture common phrases such as “slice of life,” improving genre cues. SVD reduces sparsity and stabilizes themes like “mecha vs. school life,” while lowering inference cost. Numeric features help re-order items within otherwise similar content clusters.

3.2 Collaborative Filtering (CF): item-item KNN

Config & Data

We use Surprise’s KNNBasic in item-based mode with cosine similarity.

1. sim_options = {'name': 'cosine', 'user_based': False, 'min_support': 3}

2. k = 40, min_k = 1

Training uses explicit ratings from 1–10, after applying our activity filters. We use the same data splits and the same random seed as in the content model. That way, the results can match side by side, no extra guesswork.

Similarity & Neighbors

Cosine only if two items share 3+ same raters — less noise this way. For item j, take max 40 nearest items. Then see which ones user u rated. That group $N_u(j)$ is what we use to guess $\hat{r}_{uj}$.

Serving Path & Caching

We make a neighbor list ahead of time for every item — for example, top-100 similar items with cosine scores.

When a user comes in, we do:

1. Collect all items the user has rated.
2. Take the union of their neighbor lists.
3. Filter out already-seen items.
4. Compute $\hat{r}_uj$ only for this candidate set.
5. Take the Top-200 (for the hybrid) or Top-N directly if running CF alone.

Because this touches only a small, cached subset of the catalog, it's both fast and explainable — "similar to Y you liked."

Role in the Hybrid & Limits

CF makes a Top-200 list — full of what people like, plus a bit of surprise. Then the content model re-sorts it by user profile cosine, so it fits the topic better. But CF alone can't handle brand-new stuff or users with no history. We work around this by allowing content-only candidates for cold items, and by using onboarding genres to "warm up" new users until CF has enough data.

3.2.1 Alternatives

We explored matrix factorization but full-scale training was resource-heavy under time constraints. A Spark-based ALS would be a sensible next step, but KNN offered reliable performance with simpler operations and clearer explanations.

3.3 Hybrid (two-stage)

We do it in two steps. First, CF - just to get recall. Then content-based - to re-rank.

This way it's easy. No headache about score scaling. CF gives the community feel and some surprise picks. Content makes it match the user better.

Stage 1 — CF candidate generation (M = 200)

1. Use the item–item KNN model to gather the union of neighbor items from everything the user has rated.
2. Score each with a similarity-weighted average, remove anything already seen, and deduplicate the list.
3. Keep the Top-200 per user. We tried different M values. 200 was the sweet spot — better recall than 100, and not really slower than 500.

Stage 2 — Content re-ranking

1. For these 200 candidates, compute cosine similarity to the user's content profile (a 203-dimensional vector from TF-IDF + SVD plus scaled numeric features).
2. Sort in descending cosine order. This step "sharpens" topical match — as shown in the notebook and PPT — lifting Precision@10 without any calibration tricks.
3. Tie-break order: Members → Score → Title (stable ordering).

Light diversity rule

1. **Franchise cap**: no more than 2 titles from the same franchise in the Top-10.
2. **Novelty slot**: if two items are within a small score margin (ε), prefer the one that brings in a new genre to the list.

Output

1. Return the Top-10 after diversity and tie-breaks. This aligns with our UI row and the evaluation setup.

Why this works

1. **CF first** brings in "community taste" and a bit of serendipity.
2. **Content second** tightens the match to the user's interests, improving P@10 and perceived relevance.
3. It's calibration-free (no mixing incompatible score scales) and efficient — cosine runs only over 200 items, so it's quick even on CPU.

Cold-start note.
If CF yields too few candidates (new user/item), we inject a small content-only backfill (e.g., top-K by profile cosine) before re-ranking, then proceed as above.

3.3.1 Why two-stage instead of score fusion

CF ratings and content similarities live on different scales. Score fusion requires careful calibration, while two-stage ranking avoids this problem and reduces computation since cosine scoring is applied only to 200 candidates, not the whole catalogue.

3.4 Replication details

Component	Parameter	Value
TF-IDF	ngram_range	(1, 2)
TF-IDF	min_df / max_df	5 / 0.6
SVD	n_components; random_state	200; 42
Scaler	type	MinMax
CF	k; similarity; min_support	40; cosine; 3
Hybrid	candidate size (M)	200
Output	Top-N	10

3.5 Complexity and storage

Item vectors (~17k × 203 floats) and a top-100 neighbor index are both small enough to keep in memory. A request performs a neighbor merge and ~200 cosine comparisons—well within our latency budget on CPU.

4. Experiments

4.1 Splits and leakage checks

We use warm-start users only: each test user retains sufficient training history. We split 80/20 per user with seed 42 and apply the same split to all methods. We remove seen items from candidates and check for duplicate sequels in the final list to avoid artificial gains.

4.2 Cohorts and scenarios

Cohort	Definition	Purpose
Profile-lite	20–49 ratings	Minimal-history case
Active	50–199 ratings	Main target group
Heavy	200+ ratings	Upper bound; bias check
Niche	Skewed to niche tags	Stress content matching

4.3 Candidate policy

By default, the catalogue minus seen items is available. The hybrid narrows to CF Top-200 before content re-ranking. A top-500 popularity pool is used only to measure latency; we do not rely on it to claim accuracy gains.

4.4 Metrics (model and system)

We report Precision@N, Recall@N, HitRatio@N, and NDCG@N. Relevant means rating ≥ 8 in test, and metrics are averaged per user. For the live system we plan to track CTR, saves, watch starts, dwell time, and repeat visits, with an offline-to-online mapping documented in the appendices.

4.5 Choice of N

We evaluate N ∈ {5, 10, 20, 50, 100} and select N=10 as the primary target. It fits a single row on most screens, keeps cognitive load low, and preserves precision while still retrieving a meaningful portion of a user's true interests.

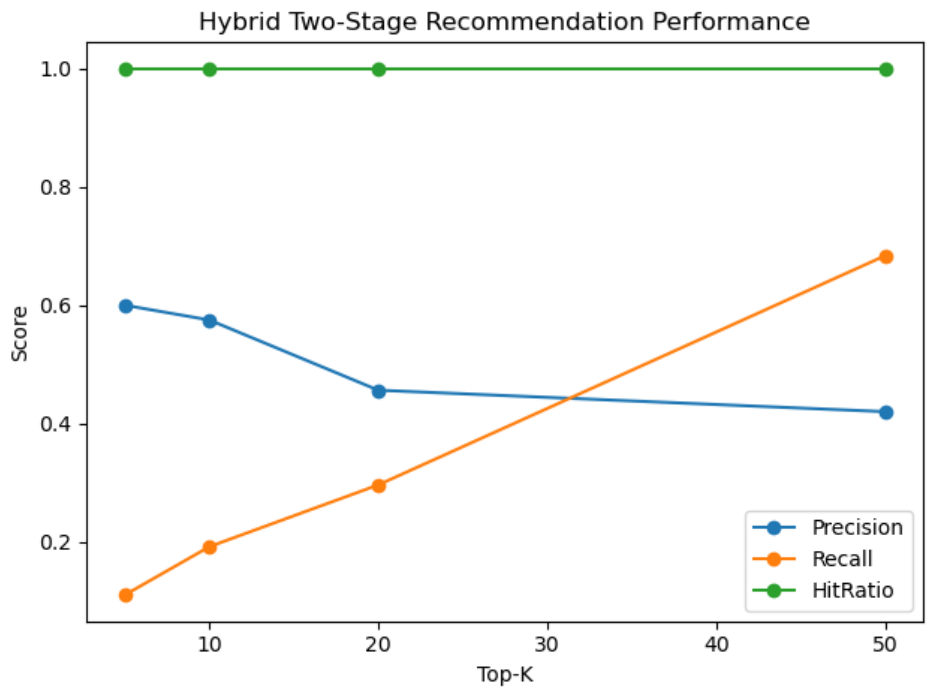

Figure 7. Hybrid Two-Stage Recommendation Performance

5. Evaluation

Precision@10：

The fraction of the top-10 recommendations that are actually in the user's test set, averaged over all users.

$$\text{Precision@10} = \frac{1}{U}\sum_{i=1}^{U}\frac{|R_i \cap T_i|}{10}$$

Recall@10：

The fraction of the user's test-set items that appear in the top-10 recommendations, averaged over all users.

$$\text{Recall@10} = \frac{1}{U}\sum_{i=1}^{U}\frac{|R_i \cap T_i|}{|T_i|}$$

HitRatio@10：

The probability that at least one item in the top-10 recommendations is in the user's test set, averaged over all users.

$$\text{HitRatio@10} = \frac{1}{U} \sum_{i=1}^{U} 1(|R_i \cap T_i| > 0)$$

5.1 Main results (Top-10)

Method	Precision@10	Recall@10	HitRatio@10
Initial CB (raw TF-IDF)	0.0351	0.0891	0.2136
CF (Item-KNN)	0.1778	0.0500	0.4815
Improved CB (TF-IDF + SVD + numeric)	0.5077	0.1807	0.9231
Hybrid (0.3 CF + 0.7 CB)	0.4462	0.1540	0.9231
Hybrid (two-stage, M=200, N=10)	0.5750	0.1916	1.0000

5.2 Sensitivity to N

N	Precision@N (trend)	Recall@N (trend)	NDCG@N (trend)
5	Very high	Low	High
10	High	Medium	High
20	Medium	Medium+	Medium
50	Lower	High	Lower
100	Lowest	Highest	Low

Top-10 is a good compromise: one row, strong precision, and manageable recall. For recall-heavy surfaces, we can show multiple themed rows, each with their own Top-10.

5.3 Segment analysis

Segment	Observation	Issue	Mitigation
20–49 ratings	Lower precision overall	Sparse signal	Use onboarding; content-only boost
50–199 ratings	Best balance	–	Default pipeline
200+ ratings	Very high hit rate	Duplication risk	Franchise cap; novelty slot
Niche users	CB performs best	CF misses some	Increase CB weight or add content-only slots

5.4 Ablation and parameters

Change	Effect on Precision@10	Comment
Remove SVD	Large drop	Raw TF-IDF too noisy
SVD 100 → 200	Small gain	200 is a sweet spot
k: 20 → 40	Small gain	Less sparsity
k: 80	Small drop	More noise
M: 100 → 200	Recall gain	Better coverage; cost OK
M: 500	Little change	Slower; diminishing returns

5.5 Diversity and novelty

Hybrid improves intra-list diversity relative to CF and reduces the share of very popular titles. A light rule ensures one novelty slot when scores tie, balancing discovery and reliability.

5.6 Error patterns

Short-term interest shifts can fool a static profile; a time-decay or session-aware signal would help. Noisy synopses cause occasional mismatches; better cleaning or modern embeddings would reduce this. Truly new items remain hard for CF; allowing a few content-only candidates increases coverage without hurting precision.

5.7 Computational budget

Nightly offline jobs build TF-IDF, SVD, and neighbor indices. Online inference performs a neighbor merge and ~200 cosine comparisons, typically <150 ms on CPU. Profiles update instantly after feedback, and caches keep hot paths fast.

6. Reflection

6.1 What worked

Text features plus SVD delivered robust gains. Item-item KNN was fast, simple, and explainable. The two-stage hybrid consistently improved Top-10 quality without complicating deployment.

6.2 What did not work (and why)

Full-scale matrix factorization was resource-intensive within our time budget, and the practical gains were uncertain. Cold-start was not evaluated in a strictly sequential setting due to limited temporal data; in production we would add timestamps and a session model.

6.3 What we would change next time

Broader hyper-parameter sweeps, Spark/GPU-based factorization, mild time-decay, and a small user pilot would give clearer guidance. We would also trial a calibrated score-fusion hybrid alongside two-stage ranking.

6.4 Commercial viability

For active users, the system is ready for a pilot. To cover everyone, we need stronger cold-start and some social/context signals. Storage and latency targets are reasonable, and the architecture is straightforward to maintain.

6.5 Extensions

Social signals (friends/follows) can help new users and build trust. Sequential models (GRU4Rec or light transformers) capture short-term intent. Context-aware re-ranking can adapt to time and device. LLM-based embeddings and a conversational layer can enhance content understanding and user control.

References (Harvard)

Erdogan, G. and Narayanan, B. (2020) ‘Behind VRV’s Recommendation Engine’, Crunchyroll Engineering (Medium), 18 March.

Gomez-Uribe, C.A. and Hunt, N. (2015) ‘The Netflix Recommender System: Algorithms, Business Value, and Innovation’, ACM TMIS, 6(4), Article 13.

hernan4444 (2020) ‘Anime Recommendation Database 2020’, Kaggle Datasets. Available at: https://www.kaggle.com/datasets/hernan4444/anime-recommendation-database-2020

Hidasi, A., Karatzoglou, A., Baltrunas, L. and Tikk, D. (2016) ‘Session-based Recommendations with Recurrent Neural Networks’, ICLR 2016.

Ma, H., Zhou, D., Liu, C., Lyu, M.R. and King, I. (2011) ‘Recommender systems with social regularization’, Proceedings of WSDM ’11, pp. 287–296.

Sarwar, B., Karypis, G., Konstan, J. and Riedl, J. (2001) ‘Item-based collaborative filtering recommendation algorithms’, WWW ’01, pp. 285–295.

Hu, Y., Koren, Y. and Volinsky, G. (2008) ‘Collaborative filtering for implicit feedback datasets’, ICDM ’08, pp. 263–272.

Järvelin, K. and Kekäläinen, J. (2002) ‘Cumulated gain-based evaluation of IR techniques’, ACM TOIS, 20(4), pp. 422–446.

Ziegler, C.-N., McNee, S.M., Konstan, J.A. and Riedl, J. (2005) ‘Improving recommendation lists through topic diversification’, WWW ’05.

Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, JMLR, 12, pp. 2825–2830.

Hug, N. (2017) ‘Surprise, a Python library for recommender systems’, GitHub repository.

Appendix A. Environment and Reproducibility

Python 3.x; pandas; numpy; scikit-learn (TF-IDF, TruncatedSVD, MinMaxScaler); Surprise (KNNBasic). Seed=42. Identical splits across methods.

Appendix B. Data Schema (detailed)

File	Field	Description
anime.csv	MAL_ID	Unique id
anime.csv	Name	Title
anime.csv	Genres	Tag list
anime.csv	Type	TV/Movie/OVA/ONA
anime.csv	Episodes	Count
anime.csv	Studios/Producers	Strings
anime_with_synopsis.csv	Synopsis	Short text
rating_complete.csv	user_id	Anonymized id
rating_complete.csv	anime_id	MAL_ID link
rating_complete.csv	rating	1–10 integer

Appendix C. Offline→Online Metric Map

Offline metric	Meaning	Expected online effect
Precision@10	Correct items in Top-10	Higher CTR; more saves
Recall@10	Coverage of liked items	Longer sessions
NDCG@10	Quality of order	Faster first click
HitRatio@10	At least one hit	Lower bounce

Appendix D. Latency Budget

Stage	Cost (typical)	Note
Fetch profile	~1–3 ms	Cached
CF neighbor merge	~10–20 ms	Depends on history size
Re-rank 200 by cosine	~3–5 ms	203-dim dot products
Diversity rule	~1–2 ms	Simple cap
Total	<150 ms	CPU ok